Take Home Assignment Solution

Prime Partners LLC

Submitted by: Nitheshkumar V

Covering: Part A – Python | Part B – React | Part C – RAG Design | Part D – README

Prime Partners LLC – Take Home Assignment Solution

Part A – Fixed Python Code

```
import sqlite3
from decimal import Decimal, ROUND_HALF_UP
from contextlib import closing


def get_invoices_for_client(client_name, db_path="invoices.db"):
    """Fetch all invoices for a given client, safely and with guaranteed cleanup."""
    with closing(sqlite3.connect(db_path)) as conn:
        cursor = conn.cursor()
        cursor.execute(
            "SELECT * FROM invoices WHERE client_name = ?",
            (client_name,)
        )
        return cursor.fetchall()


def calculate_invoice_total(line_items, tax_rate=Decimal('0.18')):
    """Sum all line item amounts, apply tax, round to nearest cent."""
    subtotal = sum(Decimal(str(item['amount'])) for item in line_items)
    total = subtotal * (Decimal('1') + tax_rate)
    return total.quantize(Decimal('0.01'), rounding=ROUND_HALF_UP)


def apply_bulk_discount(invoices, discount_percent):
    """Apply a percentage discount to each invoice's total, with validation."""
    if not (0 <= discount_percent <= 100):
        raise ValueError("discount_percent must be between 0 and 100")

    discount = Decimal(str(discount_percent))
    for invoice in invoices:
        current_total = Decimal(str(invoice['total']))
        invoice['total'] = (
            current_total - (current_total * discount / Decimal('100'))
        ).quantize(Decimal('0.01'), rounding=ROUND_HALF_UP)
    return invoices
```

Python Bugs Fixed

- **SQL Injection** — parameterized query instead of string interpolation.
- **Off-by-one** — original range(len(line_items)-1) skipped the last item; now sums every item directly.
- **Precision** — Decimal used throughout instead of float, rounded with ROUND_HALF_UP.
- **Resource leak (new)** — original code didn't close the connection if an exception occurred mid-query. Now wrapped in contextlib.closing so it always closes.
- **No input validation (new)** — apply_bulk_discount accepted any discount_percent, including negative or >100 values. Added a range check.
- **Mixed float/Decimal math (new)** — discount logic now uses Decimal end-to-end for consistency.

Part B – Fixed React Component

```
import React, { useMemo, useState } from 'react';

function InvoiceList({ invoices }) {
  const [searchTerm, setSearchTerm] = useState('');

  const filtered = useMemo(
    () =>
      invoices.filter((inv) =>
        inv.clientName.toLowerCase().includes(searchTerm.toLowerCase())
      ),
    [invoices, searchTerm]
  );

  return (
    <div>
      <input
        type="text"
        value={searchTerm}
        onChange={(e) => setSearchTerm(e.target.value)}
        placeholder="Search by client name"
      />
      <ul>
        {filtered.map((inv) => (
          <li key={inv.id}>
            {inv.clientName} - ${inv.total}
          </li>
        ))}
      </ul>
    </div>
  );
}

export default InvoiceList;
```

React Issues Fixed

- **Search lag** — filtering derived via useMemo from live searchTerm state so results update on every keystroke.
- **Missing key prop** — added key={inv.id} on each list item to prevent React reconciliation bugs.
- **Invalid JSX (new)** — original snippet had malformed/incomplete markup (likely from PDF extraction). Rewrote as valid, controlled JSX.

Part C – RAG Design

Ingestion: Chunk prose into 300–500 token semantic chunks with overlap so context isn't cut mid-sentence. Store tables separately by row/section while preserving headers. OCR scanned PDFs before indexing.

Query flow for “What did we spend with Vendor X in Q2?”

- Embed the query.
- Retrieve candidate chunks via vector search, filtered by metadata (vendor = X, date range = Q2).
- Rerank retrieved chunks for relevance.
- Aggregate invoice totals programmatically (not just via LLM reading) for accuracy.
- Generate the answer with source citations back to specific invoices.

Major failure mode: retrieving incorrect or outdated financial records (e.g., a similarly-named vendor, or a superseded invoice version).

Mitigation:

- Metadata filtering (vendor ID, date, document type, version) rather than relying on embedding similarity alone.
- Reranking to push the most relevant chunks to the top.
- Citation validation — cross-check cited invoice numbers against the retrieved set before returning an answer.
- Prefer deterministic aggregation (sum via code) over asking the LLM to add numbers from context.

Part D – README

- Fixed SQL injection with parameterized queries.
- Fixed off-by-one bug in invoice total calculation.
- Used Decimal for all finance calculations to avoid float rounding errors.
- Wrapped DB connection in contextlib.closing to prevent leaks on exception.
- Added validation to apply_bulk_discount (0–100 range check).
- Fixed React search lag by deriving filtered list from live state.
- Added React list key prop.
- Rewrote corrupted JSX into valid, working markup.
- Assumed invoices contain unique id, client_name, amount, and total fields.
- Assumed discount_percent is always provided as a plain number (int/float), not a string.